

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

**Дисциплина:
«Информационная безопасность баз данных»**

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
SQL инъекции**

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна



Проверил:
Салихов Максим Русланович

Санкт-Петербург
2025 г.

Цель работы: изучить механизмы SQL-инъекций на практике, освоить методы защиты с использованием параметризованных запросов в Python, и проанализировать уязвимости в приложениях, работающих с базами данных.

Задачи:

1. Реализовать подключение к базе данных из приложения на языке программирования python.
2. Подготовить примеры SQL-запросов, в которых пользовательские данные конкатенируются со строкой запроса.
3. Реализовать безопасный вариант выполнения SQL-запросов с использованием подготовленных выражений.

Ход работы

1. Создать БД как минимум из 2-3 отношений и заполнить ее данными (достаточно 5-6 кортежей в каждой таблице). Можно использовать отношения из прошлых лабораторных. Отношения должны быть составлены таким образом, чтобы была возможность выполнить объединение таблиц (заданы связи через внешние ключи; имелись атрибуты в таблицах, по которым возможно выполнить объединение вида inner join, left join и др).

```
CREATE DATABASE product_management;
```

```
\c product_management
```

```
CREATE EXTENSION pgcrypto;
```

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL,  
    password VARCHAR(100) NOT NULL,  
    email VARCHAR(100),  
    is_admin BOOLEAN DEFAULT false  
);
```

```
CREATE TABLE products (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL,  
    stock INTEGER NOT NULL  
);
```

```
CREATE TABLE orders (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(id),  
    product_id INTEGER REFERENCES products(id),  
    quantity INTEGER NOT NULL,  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO users (username, password, email, is_admin) VALUES  
('admin', crypt('admin123', gen_salt('bf')), 'admin@example.com', true),  
('user1', crypt('password1', gen_salt('bf')), 'user1@example.com', false),  
('user2', crypt('password2', gen_salt('bf')), 'user2@example.com', false),  
('user3', crypt('qwerty', gen_salt('bf')), 'user3@example.com', false),  
('user4', crypt('123456', gen_salt('bf')), 'user4@example.com', false);
```

```
INSERT INTO products (name, description, price, stock) VALUES
('Laptop', 'High performance laptop', 999.99, 10),
('Phone', 'Smartphone with great camera', 699.99, 15),
('Tablet', 'Lightweight tablet', 299.99, 20),
('Headphones', 'Wireless noise-canceling', 199.99, 30),
('Keyboard', 'Mechanical RGB keyboard', 149.99, 25),
('Mouse', 'Ergonomic wireless mouse', 59.99, 50);
```

```
INSERT INTO orders (user_id, product_id, quantity) VALUES
(1, 1, 1),
(2, 2, 2),
(3, 3, 1),
(4, 4, 1),
(5, 5, 1);
```

Admin заказал Laptop

User1 заказал 2 Phone

User2 заказал Tablet

User3 заказал Headphones

User4 заказал Keyboard

```
product_management=# select * from users;
Active code page: 65001
id | username | password | email | is_admin
-----+-----+-----+-----+-----
1 | admin | $2a$06$054XvUTNpAI3Z9T7.Gft3.7B52/NUlyFMi3owX5a8BqXZWfgJ3Hu2 | admin@example.com | t
2 | user1 | $2a$06$U6W5RmkVvk1k.lGduWlJ7NeQQcLVsuliJZD1tV/rvBrANw8yauaPC0 | user1@example.com | f
3 | user2 | $2a$06$9bxbkvqfL8DwMTxxI3iaa0acpu68aavx9zidsZPIeYUCJnyp2Adv0 | user2@example.com | f
4 | user3 | $2a$06$6j4oEU.4SKWi1h4Km9RuaeFKs0qkD51Mod7hEzz/bjTQVEtWbIunK | user3@example.com | f
5 | user4 | $2a$06$R7fK4KsKALaHP1LjthXhBu13Gdi1qVYBFnvF/r6gne.z7IwSRpN5. | user4@example.com | f
(5 строк)

product_management=# select * from products;
Active code page: 65001
id | name | description | price | stock
-----+-----+-----+-----+-----
1 | Laptop | High performance laptop | 999.99 | 10
2 | Phone | Smartphone with great camera | 699.99 | 15
3 | Tablet | Lightweight tablet | 299.99 | 20
4 | Headphones | Wireless noise-canceling | 199.99 | 30
5 | Keyboard | Mechanical RGB keyboard | 149.99 | 25
6 | Mouse | Ergonomic wireless mouse | 59.99 | 50
(6 строк)

product_management=# select * from orders;
Active code page: 65001
id | user_id | product_id | quantity | order_date
-----+-----+-----+-----+-----
1 | 1 | 1 | 1 | 2025-05-22 12:15:08.942993
2 | 2 | 2 | 2 | 2025-05-22 12:15:08.942993
3 | 3 | 3 | 1 | 2025-05-22 12:15:08.942993
4 | 4 | 4 | 1 | 2025-05-22 12:15:08.942993
5 | 5 | 5 | 1 | 2025-05-22 12:15:08.942993
(5 строк)
```

2. Описать и продемонстрировать один из способов взаимодействия с БД с уровня приложения. Для изучения можно выбрать любой язык программирования и любой фреймворк/ORM систему/интерфейс для доступа к базе данных. Для демонстрации функций фреймворка/ORM

системы/интерфейса для доступа к БД показать, как минимум, следующие действия с БД: выборка, вставка, удаление данных из вашей БД с помощью выбранного вами фреймворка или языка программирования. Составить как минимум 2 сложных запроса, в которых выполняется выборка/модификация данных в одних таблицах на основании данных из других таблиц.

Взаимодействие с БД будет происходить с помощью языка программирования python. Использоваться будет psycopg2 — стандартный интерфейс для работы с PostgreSQL из Python.

```
import psycopg2
```

```
def connect_to_db():
```

```
    # Установка соединения с БД
```

```
    try:
```

```
        conn = psycopg2.connect(
            dbname="product_management",
            user="postgres",
            password="nastya_521!",
            host="localhost",
            port="5433"
```

```
        )
```

```
        return conn
```

```
    except psycopg2.Error as e:
```

```
        print(f"Ошибка подключения к базе данных: {e}")
```

```
        return None
```

```
def print_users(users):
```

```
    # Красивый вывод пользователей (покупателей)
```

```
    print("{:<5} {:<15} {:<30} {:<10}".format("ID", "Username", "Email", "Is Admin"))
```

```
    print("-" * 65)
```

```
    for user in users:
```

```
        print("{:<5} {:<15} {:<30} {:<10}".format(user[0], user[1], user[3], user[4]))
```

```
def print_products(products):
```

```
    # Красивый вывод товаров
```

```
    print("{:<5} {:<15} {:<40} {:<10} {:<10}".format("ID", "Name", "Description", "Price",  
"Stock"))
```

```
    print("-" * 85)
```

```
    for product in products:
```

```
        print("{:<5} {:<15} {:<40} {:<10} {:<10}".format(
            product[0], product[1], product[2][:37] + "..." if len(product[2]) > 40 else product[2],
            product[3], product[4]))
```

```

def print_orders(orders):
    # Красивый вывод списка заказов
    print("{:<5} {:<10} {:<15} {:<10} {:<25}".format("ID", "User ID", "Product ID", "Quantity",
"Order Date"))
    print("-" * 70)
    for order in orders:
        print("{:<5} {:<10} {:<15} {:<10} {:<25}".format(
            order[0], order[1], order[2], order[3],
            str(order[4][:19]))

def demonstrate_operations():
    conn = None
    cur = None
    try:
        conn = connect_to_db()
        if not conn:
            return

        cur = conn.cursor()

        # 1. Выборка данных (SELECT)
        print("\n1. Выборка всех пользователей:")
        cur.execute("SELECT * FROM users;")
        print_users(cur.fetchall())

        # 2. Вставка данных (INSERT)
        print("\n2. Добавление нового товара:")
        new_product = ("Monitor", "27-inch 4K monitor with HDR", 399.99, 15)
        cur.execute(
            "INSERT INTO products (name, description, price, stock) VALUES (%s, %s, %s, %s)
RETURNING *",
            new_product
        )
        inserted_product = cur.fetchone()
        conn.commit()
        print("Добавлен товар:")
        print_products([inserted_product])
        print("\n")

        # Выводим полный обновлённый список
        cur.execute("SELECT * FROM products ORDER BY id")
        print("Текущий список товаров:")
        print_products(cur.fetchall())

        # 3. Обновление данных (UPDATE)
        print("\n3. Обновление цены товара:")
        cur.execute(

```

```

    "UPDATE products SET price = %s WHERE name = %s RETURNING *",
    (449.99, "Monitor")
)
updated_product = cur.fetchone()
conn.commit()
print("Обновленный товар:")
print_products([updated_product])
print("\n")

# Выводим полный обновлённый список
cur.execute("SELECT * FROM products ORDER BY id")
print("Текущий список товаров:")
print_products(cur.fetchall())

# 4. Удаление данных (DELETE)
print("\n4. Удаление товара:")
cur.execute(
    "DELETE FROM products WHERE name = %s RETURNING *",
    ("Monitor",)
)
deleted_product = cur.fetchone()
conn.commit()
print("Удален товар:")
print_products([deleted_product])
print("\n")

# Выводим полный обновлённый список
cur.execute("SELECT * FROM products ORDER BY id")
print("Текущий список товаров:")
print_products(cur.fetchall())

# 5. Сложный запрос 1: Заказы с информацией о пользователях и товарах
print("\n5. Сложный запрос 1: Детали заказов")
cur.execute("""
    SELECT o.id, u.username, p.name, o.quantity, p.price, o.order_date
    FROM orders o
    JOIN users u ON o.user_id = u.id
    JOIN products p ON o.product_id = p.id
    WHERE p.price > %s
    ORDER BY o.order_date DESC
""", (500,))

print("\nДорогие заказы (price > 500):")
print("{:<5} {:<15} {:<15} {:<10} {:<10} {:<20}".format(
    "ID", "User", "Product", "Qty", "Price", "Date"))
print("-" * 80)
for order in cur.fetchall():
    print("{:<5} {:<15} {:<15} {:<10} {:<10} {:<20}".format(

```

```

order[0], order[1], order[2], order[3], order[4], str(order[5])[:19]))

# 6. Сложный запрос 2: Анализ продаж
print("\n6. Сложный запрос 2: Анализ продаж по пользователям")
cur.execute("""
    SELECT u.username, COUNT(o.id) as order_count,
           SUM(p.price * o.quantity) as total_spent
    FROM orders o
    JOIN users u ON o.user_id = u.id
    JOIN products p ON o.product_id = p.id
    GROUP BY u.username
    ORDER BY total_spent DESC
""")

print("\nСтатистика покупок:")
print("{:<15} {:<15} {:<15}".format("User", "Orders", "Total Spent"))
print("-" * 45)
for stat in cur.fetchall():
    print("{:<15} {:<15} {:<15.2f}".format(stat[0], stat[1], stat[2]))

except psycopg2.Error as e:
    print(f"Ошибка при работе с базой данных: {e}")
finally:
    if cur:
        cur.close()
    if conn:
        conn.close()

...
if __name__ == "__main__":
    print("=" * 50)
    print("Демонстрация работы с базой данных product_management")
    print("=" * 50)

    demonstrate_operations()

...
RETURNING* возвращает только что добавленную запись.

```



```
C:\py.project\first\venv\Scripts\python.exe C:\py.project\first\is_bd_lab.py
```

```
=====
```

Демонстрация работы с базой данных product_management

```
=====
```

1. Выборка всех пользователей:

ID	Username	Email	Is Admin

1	admin	admin@example.com	1
2	user1	user1@example.com	0
3	user2	user2@example.com	0
4	user3	user3@example.com	0
5	user4	user4@example.com	0

2. Добавление нового товара:

Добавлен товар:

ID	Name	Description	Price	Stock

16	Monitor	27-inch 4K monitor with HDR	399.99	15

Текущий список товаров:

ID	Name	Description	Price	Stock

1	Laptop	High performance laptop	999.99	10
2	Phone	Smartphone with great camera	699.99	15
3	Tablet	Lightweight tablet	299.99	20
4	Headphones	Wireless noise-canceling	199.99	30
5	Keyboard	Mechanical RGB keyboard	149.99	25
6	Mouse	Ergonomic wireless mouse	59.99	50
16	Monitor	27-inch 4K monitor with HDR	399.99	15

3. Обновление цены товара:

Обновленный товар:

ID	Name	Description	Price	Stock

16	Monitor	27-inch 4K monitor with HDR	449.99	15

Текущий список товаров:

ID	Name	Description	Price	Stock

1	Laptop	High performance laptop	999.99	10
2	Phone	Smartphone with great camera	699.99	15
3	Tablet	Lightweight tablet	299.99	20
4	Headphones	Wireless noise-canceling	199.99	30
5	Keyboard	Mechanical RGB keyboard	149.99	25
6	Mouse	Ergonomic wireless mouse	59.99	50
16	Monitor	27-inch 4K monitor with HDR	449.99	15

4. Удаление товара:

Удален товар:

ID	Name	Description	Price	Stock
16	Monitor	27-inch 4K monitor with HDR	449.99	15

Текущий список товаров:

ID	Name	Description	Price	Stock
1	Laptop	High performance laptop	999.99	10
2	Phone	Smartphone with great camera	699.99	15
3	Tablet	Lightweight tablet	299.99	20
4	Headphones	Wireless noise-canceling	199.99	30
5	Keyboard	Mechanical RGB keyboard	149.99	25
6	Mouse	Ergonomic wireless mouse	59.99	50

5. Сложный запрос 1: Детали заказов

Дорогие заказы (price > 500):

ID	User	Product	Qty	Price	Date
1	admin	Laptop	1	999.99	2025-05-22 12:15:08
2	user1	Phone	2	699.99	2025-05-22 12:15:08

6. Сложный запрос 2: Анализ продаж по пользователям

Статистика покупок:

User	Orders	Total Spent
user1	1	1399.98
admin	1	999.99
user2	1	299.99
user3	1	199.99
user4	1	149.99

=====

Конец работы с БД

=====

3. Для изучения проблемы фильтрации данных подготовить пример аналогичный заданному в указаниях к данной лабораторной работе (Пример 1.). Пример может быть подготовлен на любом языке программирования. Предусмотреть в примере два случая подготовки SQL запросов (подготовленные запросы, конкатенация параметров со строкой запроса).

SQL-инъекция — это серьёзная уязвимость безопасности, при которой злоумышленник может внедрить вводимые им данные таким образом, чтобы они были интерпретированы как SQL-код.

Конкатенация строк: формирование SQL-запроса путём "склеивания" строк и пользовательских данных.

Проблемы:

1. Уязвимость к SQL-инъекциям.
2. Ошибки формата: если в данных есть кавычки — запрос сломается.

Подготовленные запросы: отделение SQL-кода от данных. Параметры передаются отдельно и автоматически экранируются.

Преимущества:

1. Защита от инъекций: параметры обрабатываются как данные, а не часть кода.
2. Читаемость: четкое разделение логики запроса и данных.
3. Производительность: запрос компилируется один раз и может выполняться многократно с разными параметрами.

То есть подготовленные запросы отделяют SQL-логику от данных, обеспечивая безопасность, стабильность и эффективность выполнения, в то время как конкатенация — это рискованное и нестабильное «склеивание» кода и ввода, уязвимое к атакам и ошибкам.

```
def unsafe_login(username, password):
    # Уязвимая функция входа (конкатенация строк)
    conn = connect_to_db()
    if not conn:
        return None

    cur = conn.cursor()
    # Опасный способ - конкатенация строк
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = "
    crypt('{password}', password)"
    print(f"\n[Уязвимый запрос] Выполняется: {query}")

    try:
        cur.execute(query)
        user = cur.fetchone()
        if user:
            print(f"Успешный вход как: {user[1]}")
        else:
            print("Ошибка: неверные данные")
        return user
    except psycopg2.Error as e:
```

```

        print(f"Ошибка запроса: {e}")
        return None
    finally:
        cur.close()
        conn.close()

def safe_login(username, password):
    # Безопасная функция входа (подготовленные запросы)
    conn = connect_to_db()
    if not conn:
        return None

    cur = conn.cursor()
    # Безопасный способ - параметры
    query = "SELECT * FROM users WHERE username = %s AND password = crypt(%s, password)"
    print(f"\n[Безопасный запрос] Выполняется: {query} с параметрами ({username}, {password})")

    try:
        cur.execute(query, (username, password))
        user = cur.fetchone()
        if user:
            print(f"Успешный вход как: {user[1]}")
        else:
            print("Ошибка: неверные данные")
        return user
    except psycopg2.Error as e:
        print(f"Ошибка запроса: {e}")
        return None
    finally:
        cur.close()
        conn.close()

...
if __name__ == "__main__":
    ...
    print("\n" + "=" * 50)
    print("Фильтрация данных")
    print("=" * 50)

    print("\nТест 1: Корректные данные")
    safe_login("admin", "admin123")
    print("\nТест 1: Некорректные данные")
    unsafe_login("admin", "admin123")

    print("\nТест 2: SQL-инъекция в уязвимую систему")
    unsafe_login("admin' --", "anything")

```

```
print("\nТест 3: SQL-инъекция в защищенную систему")
safe_login("admin' --", "anything")
...
```

```
=====
Фильтрация данных
=====

Тест 1: Корректные данные

[Безопасный запрос] Выполняется: SELECT * FROM users WHERE username = %s AND password = crypt(%s, password) с параметрами (admin, admin123)
Успешный вход как: admin

Тест 1: Некорректные данные

[Уязвимый запрос] Выполняется: SELECT * FROM users WHERE username = 'admin' AND password = crypt('admin123', password)
Успешный вход как: admin

Тест 2: SQL-инъекция в уязвимую систему

[Уязвимый запрос] Выполняется: SELECT * FROM users WHERE username = 'admin' --' AND password = crypt('anything', password)
Успешный вход как: admin

Тест 3: SQL-инъекция в защищенную систему

[Безопасный запрос] Выполняется: SELECT * FROM users WHERE username = %s AND password = crypt(%s, password) с параметрами (admin' --, anything)
Ошибка: неверные данные
```

4. Для варианта конкатенации параметров, вводимых пользователем, со строкой запроса продемонстрировать возможные варианты проведения SQL-инъекций. Например, показать, как в случае объединения таблиц злоумышленник может узнать количество столбцов второй таблицы. Предложить подход для получения структуры базы данных (включая название столбцов таблицы). Показать устойчивость или уязвимость варианта с подготовленными параметрами к выбранным вами вариантам проведения SQL-инъекций.

Объединение таблиц позволяет нам сопоставлять наши догадки с реальными данными. То есть мы составляем такие *UNION-запросы*, которые могут сработать, а могут и не сработать.

Для получения структуры БД, а также структур самих таблиц можно использовать системные таблицы *information_schema*. Они нам помогают вытащить данные, не играя при этом в «угадайку».

Также попытаемся рассмотреть, можно ли получить системную информацию (например, версию PostgreSQL). Будем делать это с помощью параметра *version()*.

```
def demonstrate_sql_injection():
    print("\n" + "=" * 50)
    print("Демонстрация SQL-инъекций с полным выполнением условий задания")
    print("=" * 50)
```

1. Определение количества столбцов через UNION

```

print("\n1. Определение количества столбцов в таблице products:")

for i in range(1, 10): # Проверяем от 1 до 9 столбцов
    conn = connect_to_db()
    if not conn:
        continue

    cur = conn.cursor()
    try:
        malicious_input = f"1 UNION SELECT {'NULL,' * (i - 1)}NULL --"
        query = f"SELECT * FROM products WHERE id = {malicious_input}"
        cur.execute(query)
        print(f"Успешно с {i} столбцами")
        cur.close()
        conn.close()
        break
    except psycopg2.Error:
        print(f"Ошибка с {i} столбцами")
    finally:
        cur.close()
        conn.close()

```

2. Эксплуатация после определения столбцов

```

conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nЭксплуатация (когда знаем количество столбцов):")
    try:
        malicious_input = "1 UNION SELECT 1,'test','test',1,1 FROM users --"
        query = f"SELECT * FROM products WHERE id = {malicious_input}"
        cur.execute(query)
        print("Результат UNION-инъекции:", cur.fetchall())
    except psycopg2.Error as e:
        print(f"Ошибка при эксплуатации: {e}")
    finally:
        cur.close()
        conn.close()

```

3. Получение структуры базы данных

```

conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\n2. Получение структуры БД:")

```

```

try:
    # 3.1. Получаем список таблиц через information_schema
    print("\nСписок таблиц в БД:")
    cur.execute("""
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema='public'
        AND table_type='BASE TABLE'
        ORDER BY table_name
    """)
    for table in cur.fetchall():
        print(f"- {table[0]}")

    # 3.2. Получаем структуру конкретной таблицы users
    print("\nСтруктура таблицы users:")
    cur.execute("""
        SELECT column_name, data_type
        FROM information_schema.columns
        WHERE table_name='users'
        ORDER BY ordinal_position
    """)
    for column in cur.fetchall():
        print(f"{column[0]} ({column[1]})")

except psycopg2.Error as e:
    print(f"Ошибка при получении структуры: {e}")
finally:
    cur.close()
    conn.close()

```

4. Сравнение уязвимого и безопасного подходов

```

print("\n3. Сравнение подходов:")

```

4.1. Уязвимый подход (конкатенация)

```

conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nУязвимый подход (конкатенация):")
    try:
        # Используем строковое поле name вместо id
        malicious_input = "'nonexistent' UNION SELECT
        NULL,version(),NULL,NULL,NULL --"
        query = f"SELECT * FROM products WHERE name = {malicious_input}"
        print(f"Запрос: {query}")
        cur.execute(query)
        result = cur.fetchall()
    
```

```

    if result:
        print("Успешная инъекция! Результат:", result[0][1])
    else:
        print("Нет результатов (но запрос выполнен)")
except psycopg2.Error as e:
    print(f"Ошибка: {e}")
finally:
    cur.close()
    conn.close()

# 4.2. Безопасный подход (параметризованные запросы)
conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nБезопасный подход (параметризованные запросы):")
    try:
        safe_query = "SELECT * FROM products WHERE name = %s"
        malicious_param = "'nonexistent' UNION SELECT
NULL,version(),NULL,NULL,NULL --"
        print(f"Запрос: {safe_query} с параметром ({malicious_param})")
        cur.execute(safe_query, (malicious_param,))
        print("Результат:", cur.fetchall()) # Должен быть пустым
        print("Инъекция не сработала")
    except psycopg2.Error as e:
        print(f"Ошибка (ожидаемо): {e}")
    finally:
        cur.close()
        conn.close()

...
if __name__ == "__main__":
    ...
    demonstrate_sql_injection()

print("\n" + "=" * 50)
print("Конец работы с БД")
print("=" * 50)

```

```

=====
Демонстрация SQL-инъекций с полным выполнением условий задания
=====

1. Определение количества столбцов в таблице products:
Ошибка с 1 столбцами
Ошибка с 2 столбцами
Ошибка с 3 столбцами
Ошибка с 4 столбцами
Успешно с 5 столбцами

Эксплуатация (когда знаем количество столбцов):
Результат UNION-инъекции: [(1, 'test', 'test', Decimal('1'), 1), (1, 'Laptop', 'High performance Laptop', Decimal('999.99'), 10)]

```


2. Получение структуры БД:

Список таблиц в БД:

- orders
- products
- users

Структура таблицы users:

```
id (integer)
username (character varying)
password (character varying)
email (character varying)
is_admin (boolean)
```

3. Сравнение подходов:

Уязвимый подход (конкатенация):

Запрос: `SELECT * FROM products WHERE name = 'nonexistent' UNION SELECT NULL,version(),NULL,NULL,NULL --`

Успешная инъекция! Результат: PostgreSQL 17.4 on x86_64-windows, compiled by msvc-19.42.34436, 64-bit

Безопасный подход (параметризованные запросы):

Запрос: `SELECT * FROM products WHERE name = %s` с параметром ('nonexistent' UNION SELECT NULL,version(),NULL,NULL,NULL --)

Результат: []

Инъекция не сработала

=====

Конец работы с БД

=====

Итоговый код на языке программирования python

```
import psycopg2
from psycopg2 import sql
from datetime import datetime

def connect_to_db():
    # Установка соединения с БД
    try:
        conn = psycopg2.connect(
            dbname="product_management",
            user="postgres",
            password="nastya_521!",
            host="localhost",
            port="5433"
        )
        return conn
    except psycopg2.Error as e:
        print(f"Ошибка подключения к базе данных: {e}")
        return None

def print_users(users):
    # Красивый вывод пользователей (покупателей)
    print("{:<5} {:<15} {:<30} {:<10}".format("ID", "Username", "Email", "Is Admin"))
    print("-" * 65)
    for user in users:
        print("{:<5} {:<15} {:<30} {:<10}".format(user[0], user[1], user[3], user[4]))

def print_products(products):
    # Красивый вывод товаров
    print("{:<5} {:<15} {:<40} {:<10} {:<10}".format("ID", "Name", "Description", "Price",
"Stock"))
    print("-" * 85)
    for product in products:
        print("{:<5} {:<15} {:<40} {:<10} {:<10}".format(
            product[0], product[1], product[2][:37] + "..." if len(product[2]) > 40 else product[2],
            product[3], product[4]))

def print_orders(orders):
    # Красивый вывод списка заказов
    print("{:<5} {:<10} {:<15} {:<10} {:<25}".format("ID", "User ID", "Product ID", "Quantity",
"Order Date"))
    print("-" * 70)
    for order in orders:
        print("{:<5} {:<10} {:<15} {:<10} {:<25}".format(
```

```
order[0], order[1], order[2], order[3],  
str(order[4][:19]))
```

```
def demonstrate_operations():  
    conn = None  
    cur = None  
    try:  
        conn = connect_to_db()  
        if not conn:  
            return  
  
        cur = conn.cursor()  
  
        # 1. Выборка данных (SELECT)  
        print("\n1. Выборка всех пользователей:")  
        cur.execute("SELECT * FROM users;")  
        print_users(cur.fetchall())  
  
        # 2. Вставка данных (INSERT)  
        print("\n2. Добавление нового товара:")  
        new_product = ("Monitor", "27-inch 4K monitor with HDR", 399.99, 15)  
        cur.execute(  
            "INSERT INTO products (name, description, price, stock) VALUES (%s, %s, %s, %s)  
RETURNING *",  
            new_product  
        )  
        inserted_product = cur.fetchone()  
        conn.commit()  
        print("Добавлен товар:")  
        print_products([inserted_product])  
        print("\n")  
  
        # Выводим полный обновлённый список  
        cur.execute("SELECT * FROM products ORDER BY id")  
        print("Текущий список товаров:")  
        print_products(cur.fetchall())  
  
        # 3. Обновление данных (UPDATE)  
        print("\n3. Обновление цены товара:")  
        cur.execute(  
            "UPDATE products SET price = %s WHERE name = %s RETURNING *",  
            (449.99, "Monitor")  
        )  
        updated_product = cur.fetchone()  
        conn.commit()  
        print("Обновленный товар:")  
        print_products([updated_product])
```

```
print("\n")
```

```
# Выводим полный обновлённый список
cur.execute("SELECT * FROM products ORDER BY id")
print("Текущий список товаров:")
print_products(cur.fetchall())
```

```
# 4. Удаление данных (DELETE)
```

```
print("\n4. Удаление товара:")
cur.execute(
    "DELETE FROM products WHERE name = %s RETURNING *",
    ("Monitor",)
)
deleted_product = cur.fetchone()
conn.commit()
print("Удален товар:")
print_products([deleted_product])
print("\n")
```

```
# Выводим полный обновлённый список
cur.execute("SELECT * FROM products ORDER BY id")
print("Текущий список товаров:")
print_products(cur.fetchall())
```

```
# 5. Сложный запрос 1: Заказы с информацией о пользователях и товарах
```

```
print("\n5. Сложный запрос 1: Детали заказов")
cur.execute("""
    SELECT o.id, u.username, p.name, o.quantity, p.price, o.order_date
    FROM orders o
    JOIN users u ON o.user_id = u.id
    JOIN products p ON o.product_id = p.id
    WHERE p.price > %s
    ORDER BY o.order_date DESC
    """, (500,))
```

```
print("\nДорогие заказы (price > 500):")
print("{:<5} {:<15} {:<15} {:<10} {:<10} {:<20}".format(
    "ID", "User", "Product", "Qty", "Price", "Date"))
print("-" * 80)
for order in cur.fetchall():
    print("{:<5} {:<15} {:<15} {:<10} {:<10} {:<20}".format(
        order[0], order[1], order[2], order[3], order[4], str(order[5])[:19]))
```

```
# 6. Сложный запрос 2: Анализ продаж
```

```
print("\n6. Сложный запрос 2: Анализ продаж по пользователям")
cur.execute("""
    SELECT u.username, COUNT(o.id) as order_count,
    SUM(p.price * o.quantity) as total_spent
```

```

FROM orders o
JOIN users u ON o.user_id = u.id
JOIN products p ON o.product_id = p.id
GROUP BY u.username
ORDER BY total_spent DESC
""")

print("\nСтатистика покупок:")
print("{:<15} {:<15} {:<15}".format("User", "Orders", "Total Spent"))
print("-" * 45)
for stat in cur.fetchall():
    print("{:<15} {:<15} {:<15.2f}".format(stat[0], stat[1], stat[2]))

except psycopg2.Error as e:
    print(f"Ошибка при работе с базой данных: {e}")
finally:
    if cur:
        cur.close()
    if conn:
        conn.close()

def unsafe_login(username, password):
    # Уязвимая функция входа (конкатенация строк)
    conn = connect_to_db()
    if not conn:
        return None

    cur = conn.cursor()
    # Опасный способ - конкатенация строк
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = "
    crypt('{password}', password)"
    print(f"\n[Уязвимый запрос] Выполняется: {query}")

    try:
        cur.execute(query)
        user = cur.fetchone()
        if user:
            print(f"Успешный вход как: {user[1]}")
        else:
            print("Ошибка: неверные данные")
        return user
    except psycopg2.Error as e:
        print(f"Ошибка запроса: {e}")
        return None
    finally:
        cur.close()
        conn.close()

```

```

def safe_login(username, password):
    # Безопасная функция входа (подготовленные запросы)
    conn = connect_to_db()
    if not conn:
        return None

    cur = conn.cursor()
    # Безопасный способ - параметры
    query = "SELECT * FROM users WHERE username = %s AND password = crypt(%s, password)"
    print(f"\n[Безопасный запрос] Выполняется: {query} с параметрами ({username}, {password})")

    try:
        cur.execute(query, (username, password))
        user = cur.fetchone()
        if user:
            print(f"Успешный вход как: {user[1]}")
        else:
            print("Ошибка: неверные данные")
        return user
    except psycopg2.Error as e:
        print(f"Ошибка запроса: {e}")
        return None
    finally:
        cur.close()
        conn.close()

```

```

def demonstrate_sql_injection():
    print("\n" + "=" * 50)
    print("Демонстрация SQL-инъекций с полным выполнением условий задания")
    print("=" * 50)

```

1. Определение количества столбцов через UNION

```

print("\n1. Определение количества столбцов в таблице products:")

```

```

for i in range(1, 10): # Проверяем от 1 до 9 столбцов
    conn = connect_to_db()
    if not conn:
        continue

    cur = conn.cursor()
    try:

```

```

        malicious_input = f'1 UNION SELECT {'NULL,' * (i - 1)}NULL --"
        query = f"SELECT * FROM products WHERE id = {malicious_input}"
        cur.execute(query)
        print(f"Успешно с {i} столбцами")
        cur.close()
        conn.close()
        break
    except psycopg2.Error:
        print(f"Ошибка с {i} столбцами")
    finally:
        cur.close()
        conn.close()

```

2. Эксплуатация после определения столбцов

```

conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nЭксплуатация (когда знаем количество столбцов):")
    try:
        malicious_input = "1 UNION SELECT 1,'test','test',1,1 FROM users --"
        query = f"SELECT * FROM products WHERE id = {malicious_input}"
        cur.execute(query)
        print("Результат UNION-инъекции:", cur.fetchall())
    except psycopg2.Error as e:
        print(f"Ошибка при эксплуатации: {e}")
    finally:
        cur.close()
        conn.close()

```

3. Получение структуры базы данных

```

conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\n2. Получение структуры БД:")

    try:
        # 3.1. Получаем список таблиц через information_schema
        print("\nСписок таблиц в БД:")
        cur.execute("""
            SELECT table_name
            FROM information_schema.tables
            WHERE table_schema='public'
            AND table_type='BASE TABLE'
            ORDER BY table_name

```

```

        """)
    for table in cur.fetchall():
        print(f"- {table[0]}")

# 3.2. Получаем структуру конкретной таблицы users
print("\nСтруктура таблицы users:")
cur.execute("""
    SELECT column_name, data_type
    FROM information_schema.columns
    WHERE table_name='users'
    ORDER BY ordinal_position
    """)
    for column in cur.fetchall():
        print(f'{{column[0]}} ({{column[1]}})')

except psycopg2.Error as e:
    print(f'Ошибка при получении структуры: {{e}}')
finally:
    cur.close()
    conn.close()

# 4. Сравнение уязвимого и безопасного подходов

print("\n3. Сравнение подходов:")

# 4.1. Уязвимый подход (конкатенация)
conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nУязвимый подход (конкатенация):")
    try:
        # Используем строковое поле name вместо id
        malicious_input = "nonexistent' UNION SELECT NULL,version(),NULL,NULL,NULL
--"
        query = f"SELECT * FROM products WHERE name = {{malicious_input}}"
        print(f"Запрос: {{query}}")
        cur.execute(query)
        result = cur.fetchall()
        if result:
            print("Успешная инъекция! Результат:", result[0][1])
        else:
            print("Нет результатов (но запрос выполнен)")
    except psycopg2.Error as e:
        print(f'Ошибка: {{e}}')
    finally:
        cur.close()
        conn.close()

```



```

# 4.2. Безопасный подход (параметризованные запросы)
conn = connect_to_db()
if conn:
    cur = conn.cursor()
    print("\nБезопасный подход (параметризованные запросы):")
    try:
        safe_query = "SELECT * FROM products WHERE name = %s"
        malicious_param = "nonexistent' UNION SELECT
NULL,version(),NULL,NULL,NULL --"
        print(f"Запрос: {safe_query} с параметром ({malicious_param})")
        cur.execute(safe_query, (malicious_param,))
        print("Результат:", cur.fetchall()) # Должен быть пустым
        print("Инкаъекция не сработала")
    except psycopg2.Error as e:
        print(f"Ошибка (ожидаемо): {e}")
    finally:
        cur.close()
        conn.close()

if __name__ == "__main__":
    print("=" * 50)
    print("Демонстрация работы с базой данных product_management")
    print("=" * 50)

    demonstrate_operations()

    print("\n" + "=" * 50)
    print("Фильтрация данных")
    print("=" * 50)

    print("\nТест 1: Корректные данные")
    safe_login("admin", "admin123")
    print("\nТест 1: Некорректные данные")
    unsafe_login("admin", "admin123")

    print("\nТест 2: SQL-инкаъекция в уязвимую систему")
    unsafe_login("admin' --", "anything")

    print("\nТест 3: SQL-инкаъекция в защищенную систему")
    safe_login("admin' --", "anything")

    demonstrate_sql_injection()

    print("\n" + "=" * 50)
    print("Конец работы с БД")
    print("=" * 50)

```

Вывод

SQL-инъекция — одна из самых распространённых уязвимостей безопасности веб-приложений. Она возникает из-за неправильной обработки пользовательского ввода, когда данные напрямую вставляются в SQL-запрос без проверки или фильтрации.

Через SQL-инъекции злоумышленник может:

- Получить доступ к конфиденциальным данным.
- Изменить или удалить информацию в базе данных.
- Обойти систему аутентификации.
- Получить структуру базы данных и имена таблиц/столбцов.

Эффективным способом защиты от SQL-инъекций является использование подготовленных запросов (prepared statements). Такие запросы разделяют SQL-логику и данные, что исключает возможность интерпретации пользовательских данных как SQL-кода.

Подготовленные параметры обеспечивают:

- Защиту от большинства видов SQL-инъекций.
- Лучшую производительность за счёт повторного использования скомпилированных запросов.
- Читаемость и поддерживаемость кода.

При тестировании различных вариантов реализации было показано, что:

- При использовании конкатенации строк система оказалась уязвимой к различным типам SQL-атак.
- При использовании подготовленных параметров все попытки инъекций были успешно блокированы.

Разработка безопасных приложений требует обязательной проверки и фильтрации всех входных данных. Это включает не только использование подготовленных запросов, но и дополнительную валидацию, ограничение прав доступа к БД, использование ORM и других механизмов безопасности.